

Reporte Ciudadano

Plataforma de reporte ciudadano con mecánica de votos comunitarios

Planteamiento inicial — Android Java + API Laravel

Planteamiento inicial · Versión 1 · Junio 2026

1. Descripción general

Propuesta del proyecto Reporte Ciudadano, una plataforma móvil que permite a los ciudadanos reportar problemas de infraestructura urbana de forma rápida y sencilla. Los estados de cada reporte son gestionados por la propia comunidad: los usuarios que se encuentran geográficamente cerca de un problema lo confirman o lo desestiman con un voto, y el sistema actúa automáticamente según el resultado. No se requieren operadores municipales para cerrar reportes.

2. Concepto: mecánica estilo Waze

El sistema se apoya en cuatro mecánicas principales:

Mecánica	Concepto	Detalle
Reporte en 2 toques	Toque largo en el mapa → menú radial → enviado	Sin formularios. Foto y descripción son opcionales y se agregan después.
Sigue ahí	Confirmar que el problema sigue existiendo	Solo usuarios dentro de 500 m pueden votar. Cada voto agranda el marcador en el mapa.
Ya se resolvió	Desestimar porque ya fue atendido	Si supera el 70% del total con mínimo 3 votos, el reporte se cierra automáticamente.
Verificado por la comunidad	Sello de validación comunitaria	Al llegar a 5 votos "Sigue ahí". Usuarios nivel Experto verifican con solo 3 votos.

3. Ciclo de vida automático de un reporte

Estado	Condición para alcanzarlo	Comportamiento en el mapa
Nuevo	Recién creado, sin votos aún	Marcador pequeño. Cualquier usuario cercano puede votar.
Verificado	5 o más votos "Sigue ahí" (3 si el creador es Experto)	Marcador grande con sello especial. Mayor visibilidad.
Resuelto	Votos "Ya se resolvió" >= 70% con al menos 3 votos totales	Marcador gris durante 2 horas y luego desaparece del mapa.

Estado	Condición para alcanzarlo	Comportamiento en el mapa
Archivado	Sin ninguna interacción en las últimas 24 horas	Se retira del mapa sin haber sido confirmado ni resuelto.

4. Actores del sistema

Actor	Descripción
Ciudadano / Usuario	Crea reportes, vota en los de otros usuarios y recibe alertas de proximidad.
Comunidad	Conjunto de usuarios que validan colectivamente un reporte mediante votos. Es el mecanismo principal de validación y cierre de reportes.
Laravel Sanctum	Gestiona la autenticación mediante tokens de acceso seguros. Emite y revoca tokens de Sanctum sin depender de ningún servicio externo.
Firebase Cloud Messaging (FCM)	Canal de transporte para notificaciones push. Recibe instrucciones del API Laravel y las entrega al dispositivo del usuario.
API Laravel	Núcleo del sistema. Gestiona autenticación con Sanctum, reportes, votos, scores, estados automáticos y despacha notificaciones push vía FCM.

5. Arquitectura de servicios externos

El cliente Android se apoya en dos servicios de Firebase con responsabilidades distintas y en el API Laravel como núcleo del sistema:

Laravel Sanctum — Autenticación

Laravel Sanctum es el sistema de autenticación integrado al API. Gestiona registro, tokens de acceso y sesiones sin depender de ningún servicio externo. Su alcance comprende:

- Registro con correo electrónico y contraseña, o con cuenta de Google (OAuth 2.0 verificado en el servidor con Laravel Socialite).
- Emisión de un token de Sanctum tras un inicio de sesión exitoso. El token se envía al cliente y acompaña cada petición al API en el header Authorization.
- Mantenimiento y revocación de sesiones por dispositivo. Cerrar sesión en un dispositivo no invalida la sesión en los demás.
- Recuperación de contraseña mediante enlace enviado al correo por el propio servidor Laravel (sin servicios externos).

Laravel Sanctum no depende de ningún servicio de Google ni externo. Valida cada token localmente y responde la pregunta: ¿quién eres y tienes permiso para esta acción?

Firebase Cloud Messaging — Transporte de notificaciones

FCM es el canal de entrega de notificaciones push al dispositivo Android. Su rol es exclusivamente de transporte:

- El API Laravel decide cuándo enviar una notificación (proximidad, cambio de estado, umbral de votos alcanzado).
- Laravel construye el payload de la notificación y lo envía a FCM mediante la API HTTP de FCM.
- FCM lo entrega al dispositivo del usuario identificado por su FCM token.
- FCM no conoce la lógica de negocio; solo transporta el mensaje que Laravel le instruye enviar.

FCM es el canal de transporte de notificaciones push al dispositivo Android. El API Laravel controla por completo cuándo y qué notificaciones enviar; el cliente Android no se comunica directamente con FCM.

API Laravel — Lógica de negocio

Laravel es el núcleo del sistema. Concentra toda la lógica de negocio y es el único componente que escribe y lee la base de datos de la aplicación:

- Gestiona el registro e inicio de sesión con Sanctum. Para Google OAuth: verifica el `id_token` del cliente con la librería oficial de Google, sin llamadas a Firebase.
- Emite tokens de Sanctum al autenticar exitosamente y los valida en cada petición de forma local, sin consultar ningún servicio externo.
- Gestiona reportes, votos, umbrales de cierre automático y cambios de estado.
- Calcula scores y niveles de usuario.
- Valida el radio de 500 m server-side antes de aceptar un voto.
- Resuelve concurrencia de votos simultáneos con bloqueo optimista.
- Despacha instrucciones de notificación push a FCM cuando corresponde.

Flujo de autenticación y primer uso

Paso	Actor	Acción
1	Usuario	Se registra o inicia sesión en la app.
2	API Laravel	Recibe las credenciales del cliente. Valida correo/contraseña o el <code>id_token</code> de Google. Crea o recupera el usuario en la base de datos y emite un token de Sanctum.
3	App Android	Almacena el token de Sanctum en <code>EncryptedSharedPreferences</code> .
4	App Android	Incluye el token de Sanctum en el header <code>Authorization</code> de cada petición subsiguiente al API.
5	API Laravel	Valida el token de Sanctum. Retorna el perfil del usuario (nivel, score, estadísticas).
6	API Laravel	Devuelve el perfil del usuario (nivel, score, estadísticas).
7	App Android	Laravel valida el token de Sanctum localmente en cada

Pa so	Actor	Acción
		petición sin consultar ningún servicio externo.

Flujo de notificación push

Pa so	Actor	Acción
1	API Laravel	Detecta que un reporte alcanzó un umbral (votos, proximidad, cambio de estado).
2	API Laravel	Construye el payload de la notificación con categoría, votos actuales y botones de acción.
3	API Laravel	Envía el payload a FCM vía HTTP usando el FCM token del dispositivo destino.
4	FCM	Entrega la notificación al dispositivo Android del usuario.
5	App Android	Muestra la notificación con los botones "Sigue ahí" / "Ya se resolvió".
6	Usuario	Vota directamente desde la notificación sin abrir la app.
7	App Android	Envía el voto a Laravel con el token de Sanctum del usuario. Laravel lo procesa normalmente.

6. Módulo de autenticación

La autenticación define la identidad del usuario en el sistema de votos y puntuación. Se implementa con Laravel Sanctum como sistema de autenticación integrado al API. No depende de Firebase ni de ningún servicio externo para validar identidades.

Flujo de acceso

- Primer ingreso: registro con correo electrónico y contraseña, o acceso con cuenta de Google (OAuth 2.0).
- Inicio de sesión con sesión persistente (token de Sanctum almacenado en EncryptedSharedPreferences).
- Recuperación de contraseña vía correo electrónico gestionada por el API Laravel.
- Modo invitado: el usuario puede explorar el mapa sin autenticarse, pero no puede crear reportes ni votar.

Requerimientos funcionales — Autenticación

ID	Requerimiento
RF-A01	El usuario puede registrarse con correo electrónico y contraseña, o con su cuenta de Google.
RF-A02	La sesión se mantiene activa entre reinicios de la aplicación mediante el token de Sanctum almacenado en EncryptedSharedPreferences.
RF-A03	El usuario puede recuperar su contraseña mediante enlace enviado a su correo (gestionado por el API Laravel mediante SMTP).
RF-A04	El modo invitado permite ver el mapa y los reportes, pero bloquea la creación de reportes y el voto con respuesta 401 del servidor.
RF-A05	El perfil de usuario (nombre, avatar) se crea automáticamente en el primer inicio de sesión y se sincroniza con el API Laravel.
RF-A06	El usuario puede cerrar sesión desde el perfil; el cierre elimina el token de Sanctum del dispositivo y lo revoca en el servidor Laravel.

7. Categorías de reporte

Las categorías determinan los íconos del menú radial, los filtros del mapa y las preferencias de alertas de proximidad. Se definen las siguientes ocho categorías:

#	Categoría	Ejemplos de problema
1	Vialidad	Bache, grieta, hundimiento, señalización dañada
2	Alumbrado público	Lámpara apagada, poste caído, cable expuesto
3	Recolección de basura	Montón de basura, contenedor desbordado
4	Agua y drenaje	Fuga de agua, drenaje tapado, inundación
5	Áreas verdes	Árbol caído, poda urgente, área vandalizada
6	Seguridad vial	Semáforo dañado, paso peatonal borrado
7	Infraestructura	Puente dañado, banqueta rota, rampa obstruida
8	Otro	Cualquier problema que no encaje en las categorías anteriores

El menú radial muestra máximo 8 íconos. Si en el futuro se agregan categorías, las adicionales se agrupan en un ícono "Más".

8. Requerimientos funcionales

Módulo 1 — Reporte ultrarrápido

ID	Requerimiento
RF-01	Toque largo en el mapa despliega un menú radial con los íconos de cada

ID	Requerimiento
	categoría definida en la sección 7.
RF-02	Un toque en la categoría envía el reporte de inmediato sin ningún formulario adicional.
RF-03	El GPS se captura del punto exacto tocado en el mapa, sin paso de confirmación adicional.
RF-04	La foto y descripción son opcionales y se pueden agregar desde la tarjeta del reporte ya creado.
RF-05	Si no hay conexión, el reporte se guarda en Room DB y WorkManager lo sincroniza al recuperar internet.
RF-06	El usuario puede retirar su propio reporte dentro de los primeros 5 minutos si tiene menos de 3 votos.

Módulo 2 — Sistema de votos comunitarios

ID	Requerimiento
RF-07	Solo los usuarios que se encuentran dentro de 500 m del reporte pueden emitir un voto.
RF-08	Los dos votos disponibles son: "Sigue ahí" (confirmar el problema) y "Ya se resolvió" (desestimar).
RF-09	Cada usuario emite un único voto por reporte; puede cambiarlo hasta 5 minutos después de haberlo emitido.
RF-10	El conteo se actualiza en tiempo real: "8 confirman · 3 dicen que ya se resolvió".
RF-11	Cuando votos "Ya se resolvió" superan el 70% del total con mínimo 3 votos, el reporte se cierra automáticamente.
RF-12	Al llegar a 5 votos "Sigue ahí", el reporte recibe el sello "Verificado por la comunidad" y un ícono especial en el mapa.
RF-13	Si no hay ninguna interacción en 24 horas, el reporte se archiva automáticamente como no verificado.
RF-14	El creador del reporte puede votar "Ya se resolvió" en su propio reporte para cerrarlo manualmente.
RF-15	El servidor valida la concurrencia de votos simultáneos mediante bloqueo optimista; el cliente muestra el conteo actualizado tras recibir la respuesta del servidor.

Módulo 3 — Mapa en vivo

ID	Requerimiento
RF-16	Los marcadores aumentan de tamaño proporcionalmente al número de

ID	Requerimiento
	votos "Sigue ahí" acumulados.
RF-17	Los reportes verificados (5+ confirmaciones) muestran un ícono distinto y de mayor tamaño en el mapa.
RF-18	Los reportes resueltos aparecen en gris durante 2 horas antes de desaparecer del mapa.
RF-19	El usuario puede filtrar el mapa por categoría, estado y antigüedad: última 1 h, 6 h o 24 h.
RF-20	Al tocar un marcador se muestra la tarjeta del reporte con: categoría, votos actuales, foto y botones de voto.

Módulo 4 — Alertas de proximidad

ID	Requerimiento
RF-21	Al acercarse a menos de 300 m de un reporte activo el usuario recibe una notificación push entregada vía Firebase Cloud Messaging.
RF-22	La notificación incluye los botones "Sigue ahí" y "Ya se resolvió" para votar sin necesidad de abrir la app.
RF-23	El usuario configura qué categorías de alertas desea recibir y el radio de notificación (100 m a 500 m).
RF-24	Las alertas solo se activan si el sistema detecta que el usuario está en movimiento (ActivityRecognition API).
RF-25	No se envía más de una alerta del mismo reporte al mismo usuario en un período de 2 horas.

Módulo 5 — Puntuación y confiabilidad

ID	Requerimiento
RF-26	Cada usuario tiene un score de confiabilidad que crece cuando sus reportes y votos resultan acertados.
RF-27	Puntos por acción: reporte confirmado por la comunidad +10 pts, voto "Sigue ahí" acertado +2 pts, voto "Ya se resolvió" acertado +5 pts.
RF-28	Los reportes de usuarios con score alto tienen mayor tamaño visual en el mapa desde el momento de creación.
RF-29	Niveles: Nuevo (0–50 pts), Colaborador (51–200 pts), Guardián (201–500 pts), Experto (500+ pts).
RF-30	Los usuarios nivel Experto verifican un reporte con solo 3 votos "Sigue ahí" en lugar de los 5 normales.

Módulo 6 — Perfil e historial

ID	Requerimiento
RF-31	El usuario ve todos sus reportes con el resultado comunitario: votos recibidos, sello y estado final.
RF-32	El usuario ve su historial de votos y el porcentaje de acierto (accuracy %) por tipo de voto.
RF-33	El perfil muestra estadísticas de contribución: reportes creados, confirmaciones recibidas y problemas resueltos.

Módulo 7 — Onboarding

La primera vez que el usuario completa el registro, se muestra un tutorial de 3 pantallas que explica la mecánica central antes de acceder al mapa.

Pantalla	Título	Contenido
1 / 3	Reporta en 2 toques	Ilustración del menú radial. Instrucción: mantén presionado el mapa en el lugar del problema y selecciona la categoría.
2 / 3	La comunidad decide	Ilustración de marcador con votos. Instrucción: acércate a menos de 500 m y vota si el problema sigue ahí o ya se resolvió.
3 / 3	Gana confiabilidad	Ilustración de niveles. Instrucción: cuanto más aciertos, mayor será tu nivel y más peso tendrán tus votos.

ID	Requerimiento
RF-34	El onboarding se muestra automáticamente la primera vez que el usuario completa el registro.
RF-35	El usuario puede saltar el onboarding en cualquier momento con el botón "Omitir".
RF-36	El onboarding puede revisarse desde el menú de ajustes en cualquier momento posterior.
RF-37	El estado "onboarding completado" se guarda en SharedPreferences y no se vuelve a mostrar automáticamente.

9. Requerimientos no funcionales

ID	Requerimiento
RNF-01	Android 7.0 (API 24) o superior.

ID	Requerimiento
RNF-02	El mapa carga en menos de 3 segundos en red 4G.
RNF-03	El reporte puede enviarse sin conexión y se sincroniza automáticamente con WorkManager al recuperar internet.
RNF-04	La validación del radio de 500 m para votar se realiza en el servidor, no en el cliente.
RNF-05	El estado del reporte se actualiza en menos de 5 segundos tras alcanzar el umbral de votos.
RNF-06	Las alertas de proximidad solo se activan con permiso de ubicación en primer plano concedido por el usuario.
RNF-07	Toda comunicación cliente-servidor usa HTTPS con TLS 1.2 o superior.
RNF-08	El conteo y peso de votos se calculan en el servidor para evitar manipulación desde el cliente.
RNF-09	Los tokens de Sanctum se almacenan en EncryptedSharedPreferences; nunca en texto plano ni en SharedPreferences convencionales.
RNF-10	El servidor maneja concurrencia de votos simultáneos con bloqueo optimista; el cliente no aplica lógica de resolución de conflictos.
RNF-11	Las notificaciones push se entregan exclusivamente mediante Firebase Cloud Messaging. El cliente registra su FCM token en el API Laravel al iniciar sesión.

10. Niveles de usuario

El nivel determina el peso de los votos del usuario y el umbral necesario para verificar sus reportes.

Nivel	Puntos requeridos	Votos para verificar	Beneficio principal
Nuevo	0 – 50 pts	5 votos	Acceso a todas las funciones básicas
Colaborador	51 – 200 pts	5 votos	Marcadores propios con mayor tamaño base en el mapa
Guardián	201 – 500 pts	4 votos	Votos con peso 1.25× en el sistema de validación
Experto	500+ pts	3 votos	Votos con peso 1.5×. Verificación más rápida y mayor visibilidad.